



Target Rescue

Alessandra Benassi (mat. 256179)

Giulia Modenese (mat. 268167)

Issue: Robot Planning Final Project Report

Date: September 2, 2026

University of Trento
DISI Department

New sections of the report and modifications are highlighted.

Contents

1	Updates	2
1.1	Bug fixes	2
1.2	Other changes	2
1.3	RRT and RRT* growth loop change	2
1.4	RRG - Rapidly exploring Random Graph	3
1.5	New experimental runs	3
1.6	Results and discussion	4
1.7	Appendix - Images from the simulations	5
2	Introduction	15
2.1	Project Overview	15
2.2	Scenario	15
2.3	Planning approaches	15
3	Problem Formulation	16
3.1	Workspace and robot model	16
3.2	Victim selection as an Orienteering Problem	16
3.3	Planning pipeline	16
4	Software Architecture	17
4.1	The Planner interface	17
4.2	The WorldModel	17
4.3	Shared planning pipeline	18
4.4	ROS nodes	18
5	Planning Strategies	18
5.1	Shared collision checking	19
5.2	Combinatorial planners	19
5.2.1	Visibility graph	19
5.2.2	Cell decomposition	19
5.2.3	Generalized Voronoi Diagram (GVD)	20
5.3	Sampling-based planners	20
5.3.1	Probabilistic Roadmap (PRM)	20
5.3.2	Rapidly-exploring Random Tree (RRT)	20
5.3.3	Rapidly-exploring Random Tree Star (RRT*)	20
5.4	Planner comparison	21
6	Mission Planning: the Orienteering Problem	21
6.1	Distance matrix computation	21
6.2	Exact solver	21
6.3	Greedy heuristic	22
6.4	Waypoint reconstruction	22
7	Trajectory Generation: Heading Optimization and Dubins Paths	22
7.1	Dubins paths	22
7.2	Heading optimization	22
7.3	Collision verification	23
7.4	Trajectory publication	23

8	Implementation Details	23
8.1	Package organization	23
8.2	Configuration parameters	24
8.3	Benchmarking	25
8.4	Testing	25
9	Experimental Setup	25
9.1	Simulation environment	25
9.2	Evaluation metrics	25
9.3	Experimental protocol	25
10	Results	26
11	Discussion	29
11.1	Path Quality, Completeness and Safety	29
11.2	Deterministic and Sampling-Based Planning	29
11.3	Influence of Resolution and Sample Density	31
11.4	Optimality and Computational Cost in tree-based planners	31
11.5	Limitations and Future Works	31

1 Updates

This section was added at the beginning of the original report to document the work added on the Target Rescue project. The focus was on some minor bug fixes, a change and comparison to how the RRT and RRT* growth loop decides when to stop sampling, linked to this the addition of a new planner (RRG), and a larger and more statistically relevant experiment runs.

1.1 Bug fixes

The mission specification requires the robot to reach the exit gate with the correct position and the gate's final orientation. To do so, the planner's pipeline reads the gate's quaternion from the `WorldModel`, converts it to a yaw with the `yawFromQuat` helper function, and passes that yaw as the fixed final state of the heading optimization DP described in Section 6.2. However, the sampling pipeline, in `task/reference_generation.cpp`, did not do this: it called `comb::optimizeHeadings` with the final heading hardcoded to zero, so discarding what the gate actually required.

The second bug fix was made in the RRT and RRT* tree growth. The tree root was being placed at the hardcoded position (0, 0) instead of the robot's actual start pose, so on any map where the robot did not start at the origin the entire tree was rooted in the wrong place. To avoid inconsistencies, both planners now root at `world.start.x/y`.

1.2 Other changes

The benchmark node was updated by adding an output CSV path and a run tag column, configurable as ROS parameters (e.g. `roslaunch rescue_planner benchmark.launch planner_type:=prm run_tag:=prm_nodes_300 output_csv:=node_modalities.csv`). The CSV path is not anymore hardcoded, so that different experiments, such as the ones reported in this section, can be routed to separate files or tagged apart without recompiling or changing manually the file access properties each time.

1.3 RRT and RRT* growth loop change

A behavioural change to the growth loop's stopping condition was added and tested, while also maintaining the old behaviour.

Both files now build a list of mission targets (every victim + the gate) and track, after each node insertion, if a tree node already is within a goal tolerance radius of each target with a collision free connecting segment.

At each iteration, both planners first create the list of the not connected targets, called **pending**. Usually a random point is sampled uniformly from the free workspace. However, with probability `goal_bias = 0.1`, the planner instead selects a random target directly from the **pending** list. This goal bias helps the planner connect all the targets in a reasonable number of iterations, so without reaching the set maximum number of nodes. With only uniform random sampling, the tree would need to randomly grow close enough to each victim to enter its `goal_tolerance` region. This can take many iterations, especially when there are many victims and only a few remain unconnected. The 10% goal directed sampling instead directs the tree towards the remaining targets, progressively reducing the **pending** list as the tree grows.

The growth loop's condition can be chosen by moving the commented line, as only one of the two is ever active at a time:

```
1 while(tree.size() < max_nodes && !allReached()){ // early stop once every target is
   connected
2 \\while(tree.size() < max_nodes){ // grow to the fixed node budget regardless
```

Which one was active for a given experiment is visible directly in the benchmark CSVs' nodes column: in the mission duration and victim count experiments, RRT and RRT*'s node counts vary from run to run, confirming the early stop version is active; in the node count experiment every run reaches the requested number of nodes. This choice was necessary since the core of the experiment was to control the sample budget as the independent variable.

On the other side, the early stop version is motivated by the same limitation noted for RRT/RRT* above: without it, the RRT or RRT* left to grow to a large fixed node count regardless of whether the mission is already solvable is, in effect, sampling a dense random geometric graph and then only keeping an arbitrary tree shaped subgraph of it, so its coverage of the free space starts to resemble PRM's, while its edge count stays far sparser with parent-child links only. Stopping as soon as every victim and the gate are reachable is what keeps the RRT/RRT* comparison representative of goal directed tree growth and also what makes RRT and RRT* considerably cheaper than PRM and RRG at the mission duration and victim count budgets in Tables 5 and 3, where their mean node counts (100–270) are under PRM and RRG's fixed 500 and 1500.

1.4 RRG - Rapidly exploring Random Graph

RRG reuses RRT's structure, root planting, goal biased sampling, and collision checked steering. The difference is that whenever a new node is added, it is also connected to every existing node within a shrinking connection radius: $r_n = \min(\gamma(\log n/n)^{(1/d)}, \eta)$

where n is the current node count, $d = 2$ for the planar workspace, η is the fixed steer step (here 1.0 m, matching the longest edge the tree can create), and γ is set to 1.5 times the theoretical minimum γ^* , that ensures the asymptotic connectivity of the random geometric graph analysis [3]. γ^* depends on the free space measure, which was approximated with the map border polygon's area ignoring the obstacles. This over estimates the true free space, which makes r_n slightly larger than required, so that it's conservative and not unsafe. The result is that RRG's roadmap is a graph rather than a tree, similarly to PRM. In this way, Dijkstra has real alternative routes to choose between, unlike RRT and RRT*, whose `buildRoadmapGraph()` only ever creates parent-child edges, so a shortest path search is just a rooted traversal of a structure with exactly one path between two nodes.

1.5 New experimental runs

As seen by the previous experiments, the number of victims wasn't sufficient to test the behaviour of the Orienteering solver's victim selection decision, and three repetitions weren't enough to distinguish a planner's real behaviour from a single unlucky run. For these reasons, in the following experiments the map was extended to fifteen fixed victims, an obstacle was added, and every configuration was repeated five times rather than three.

Three types of experiments were carried out: a time budget, a victim count, and a node count rescue mission. In this case, the first one, due to increased number of victims, also the mission

durations were rescaled from the original 40/70/140 s budgets to 90/150/230 s to match the larger map’s longer achievable travel distances.

The victim count experiment at fifteen and twenty victims, deliberately tries to test the Orienteering solver’s exact/heuristic threshold: the solver’s `op_method=auto` selector uses the exact Held-Karp dynamic program for at most sixteen victims and falls back to the greedy insertion plus 2-opt heuristic above that. So the fifteen victim configuration still uses the exact solver while the twenty victim configuration uses the heuristic.

The last one, is an extension of the previous experiment, to include RRG at 300 and 2000 sampled nodes.

1.6 Results and discussion

Roadmap structure and planning time affect which victims get rescued. In the previous experiments, all planners collected the same number of victims and obtained the same total score for each mission budget. The main differences were therefore limited to path length and planning time. With the new experiments, however, this is no longer true for the tree based planners. At every mission budget tested, PRM and RRG collect all the victims allowed by the available budget, while RRT and RRT* collect fewer. This is caused by the fact that the same Orienteering solving code, exact Held-Karp at $n=15$ and greedy + 2-opt at $n=20$ (Sections 5.2–5.3), runs against a different distance matrix for each planner. A tree contains only one path between any two of its nodes. Therefore, the distance reported by RRT and RRT* between two points of interest depends on the particular route created by the tree, which can contain significant detours. On average, these distances are longer than those available in the denser PRM and RRG roadmaps. As more victims are added, these longer distances make the Orienteering Problem exclude more victims to respect the travel budget.

Nodes	Planner	Plan (ms)	Edges	Path (m)	Victims	Score
300	PRM	180.1 ± 5.0	3819 ± 140	36.40 ± 0.47	10.00 ± 0.00	2340 ± 0
	RRT	168.2 ± 9.6	598	37.31 ± 1.01	9.80 ± 0.75	2136 ± 79
	RRT*	178.8 ± 12.3	598	36.67 ± 1.06	9.60 ± 0.80	2124 ± 46
	RRG	172.6 ± 7.2	1269 ± 34	36.86 ± 0.72	10.40 ± 0.49	2254 ± 105
2000	PRM	1049.1 ± 19.6	$171\,504 \pm 817$	35.17 ± 0.02	10.00 ± 0.00	2340 ± 0
	RRT	236.6 ± 4.3	3998	36.93 ± 1.04	10.20 ± 0.75	2292 ± 63
	RRT*	347.6 ± 10.9	3998	38.13 ± 0.17	11.00 ± 0.00	2293 ± 6
	RRG	393.8 ± 8.6	$47\,257 \pm 303$	35.22 ± 0.03	10.00 ± 0.00	2340 ± 0

Table 1: Node experiment with fixed 150 s budget: mean $\pm$ population standard deviation over 5 runs

Table 3 shows that a low budget makes the victim selection more important, but the quality of the roadmap still has a larger effect than the choice between the exact and greedy Orienteering solvers. At 130s, the map with fifteen victims allows at most nine victims to be collected, while at 200s the map with twenty victims allows at most sixteen. In this way, the robot can’t visit every victim, and the planner must make meaningful choices about which victims to include.

To compare the results, each planner’s collected score can be measured relative to the deterministic Visibility planner result for the same map: 2065 for $n = 15$ and 3595 for $n = 20$. PRM and RRG are very close to this reference, while RRT and RRT* perform less.

The main ordering therefore remains unchanged: PRM and RRG produce results close to the deterministic reference, while RRT and RRT* collect significantly less value due to the reduced number of connections. This is the same roadmap quality effect observed in the mission duration experiment in Table 2, and it appears both when using the exact Held-Karp DP and when using the greedy + 2-opt heuristic. None of the planners showed a consistent loss at $n = 20$, which would be expected if switching to the heuristic solver introduced a degradation in solution quality.

Budget	Planner	Plan (ms)	Nodes	Edges	Path (m)	Victims (of 15)	Score
90 s	PRM	184.9 ± 4.6	500	$10\,604 \pm 209$	21.62 ± 0.09	5.00 ± 0.00	1285 ± 0
	RRT	125.6 ± 11.4	193 ± 17	383 ± 35	21.75 ± 1.27	3.60 ± 1.36	716 ± 135
	RRT*	124.7 ± 6.2	212 ± 23	423 ± 45	22.06 ± 0.79	4.00 ± 1.10	904 ± 83
	RRG	236.1 ± 33.3	1500	$26\,528 \pm 218$	21.53 ± 0.12	5.00 ± 0.00	1285 ± 0
150 s	PRM	188.8 ± 25.2	500	$10\,728 \pm 304$	35.90 ± 0.34	10.00 ± 0.00	2340 ± 0
	RRT	153.0 ± 10.8	201 ± 33	400 ± 66	36.46 ± 1.10	8.60 ± 1.36	1978 ± 108
	RRT*	200.3 ± 45.0	171 ± 16	340 ± 32	37.84 ± 0.25	9.00 ± 0.63	2028 ± 102
	RRG	302.5 ± 5.8	1500	$26\,556 \pm 239$	35.33 ± 0.08	10.00 ± 0.00	2340 ± 0
230 s	PRM	267.5 ± 8.4	500	$10\,731 \pm 236$	53.42 ± 0.58	15.00 ± 0.00	2890 ± 0
	RRT	185.9 ± 10.2	194 ± 12	386 ± 24	55.74 ± 2.04	12.40 ± 0.80	2547 ± 79
	RRT*	189.7 ± 9.0	192 ± 17	382 ± 35	57.67 ± 0.89	12.80 ± 1.17	2594 ± 143
	RRG	348.4 ± 7.4	1500	$27\,090 \pm 323$	52.45 ± 0.12	15.00 ± 0.00	2890 ± 0

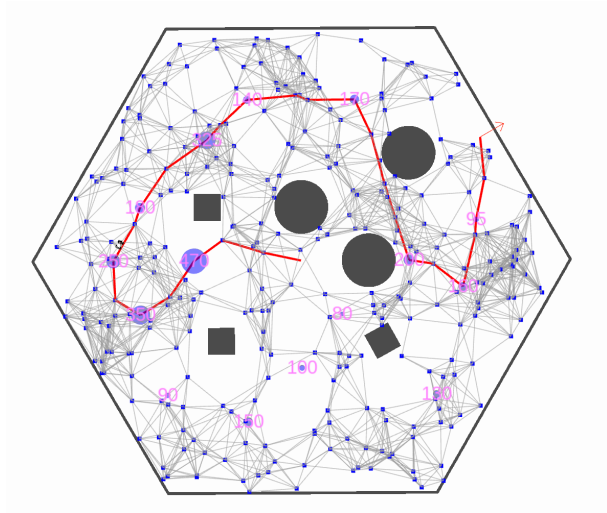
Table 2: Mission duration experiment on the 15 victims map

One interesting result is that RRG collects a slightly higher score than the Visibility planner for $n = 15$ (2085 compared with 2065), even though the Visibility roadmap produces the shortest collision-free paths between points of interest. The explanation is related to how the travel budget $D_{\max}$ is adjusted. All planners initially use the same nominal budget, based on the mission duration T and the same planning parameters. After the Orienteering solver selects a tour, however, each planner generates its own Dubins trajectory and checks if the resulting trajectory can be completed within the available mission time. If the trajectory is too long, $D_{\max}$ is reduced and the Orienteering Problem is solved again. This correction is performed independently by each planner. As a result, even though all planners start with the same budget, they can eventually use slightly different final budgets. The size of the correction depends on the additional distance introduced when the roadmap path is converted into a flyable Dubins trajectory. A path with well aligned segments (e.g. PRM) requires less additional distance than a path containing sharper changes in direction (e.g. RRT), that requires a larger correction. In this case, RRG may have required a smaller correction than the Visibility planner and therefore retained a slightly larger final budget. This could allow it to select a different eight victim subset with a higher total score. In conclusion, the result does not mean that RRG found a better route than Visibility. The Visibility planner’s Held-Karp solution is still optimal and the two planners were solving the Orienteering Problem with slightly different effective travel budgets.

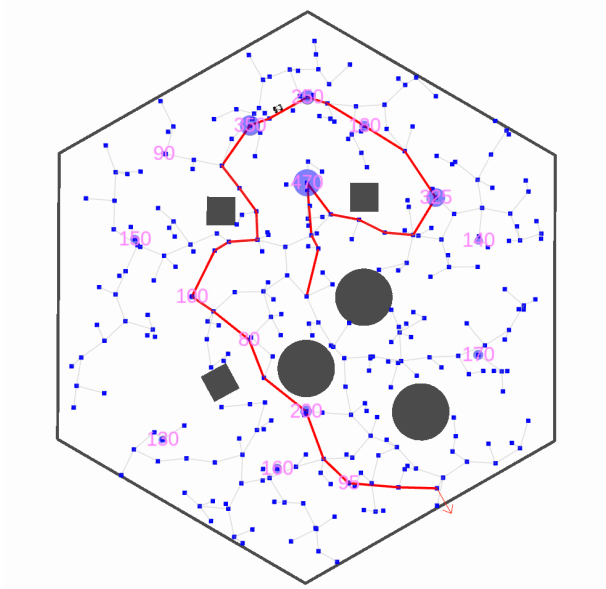
RRG was found particularly efficient at higher node counts because its connection strategy produces a much sparser graph than PRM. In the node count experiment, growing the planners to 2000 nodes requires approximately 1.05 s of planning time for PRM, compared with 0.39 s for RRG. At the same number of nodes, PRM’s fixed connection radius creates a much denser graph (172000 edges vs RRG’s 47000) while providing the same victim coverage.

RRT and RRT* remain cheaper still at 2000 nodes (0.24s and 0.35 s respectively) because they only add parent-child connections. However, this sparse tree structure is also the reason why their distance estimates are less reliable: since only one route exists between two connected nodes, the planner cannot choose between alternative paths to find a shorter connection.

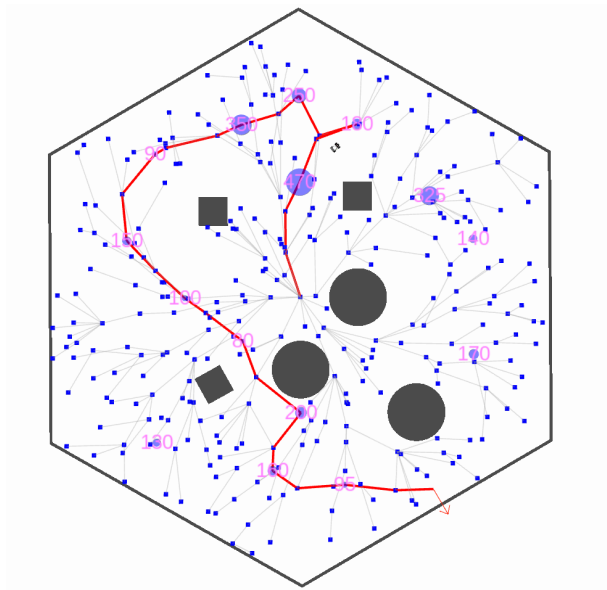
1.7 Appendix - Images from the simulations



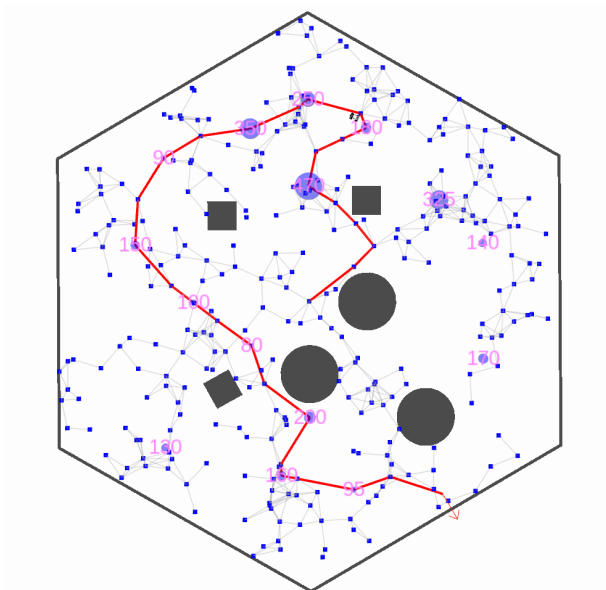
(a) PRM, 300 nodes



(b) RRT, 300 nodes



(c) RRT*, 300 nodes



(d) RRG, 300 nodes

Figure 1: Comparison of roadmaps depending on the number of nodes – 300 nodes

Map / budget	Planner	Plan (ms)	Nodes	Edges	Path (m)	Victims rescued	Score
15 victims	PRM	202.0 $\pm$ 5.9	500	10 809 $\pm$ 141	32.82 $\pm$ 0.23	8.00 $\pm$ 0.00	2063 $\pm$ 27
	RRT	140.3 $\pm$ 11.0	199 $\pm$ 26	396 $\pm$ 53	32.19 $\pm$ 0.57	7.40 $\pm$ 1.36	1687 $\pm$ 219
	RRT*	150.5 $\pm$ 7.4	186 $\pm$ 25	369 $\pm$ 51	32.67 $\pm$ 0.33	7.60 $\pm$ 1.20	1677 $\pm$ 165
	RRG	284.7 $\pm$ 5.6	1500	26 836 $\pm$ 247	32.82 $\pm$ 0.07	8.00 $\pm$ 0.00	2085 $\pm$ 0
	Visibility	281.3	29	193	33.43	9.00	2065
	Cell decomp. (exact)	367.7	80	81	33.43	9.00	2065
	Voronoi / GVD	1407.1	1915	2838	33.44	9.00	2065
	Cell decomp. (approx.)	3719.5	21 274	41 905	33.05	8.00	2085
20 victims	PRM	194.4 $\pm$ 10.0	500	10 664 $\pm$ 191	45.35 $\pm$ 0.74	15.20 $\pm$ 0.40	3463 $\pm$ 88
	RRT	115.7 $\pm$ 12.0	209 $\pm$ 31	416 $\pm$ 62	46.88 $\pm$ 3.96	13.60 $\pm$ 1.20	3126 $\pm$ 265
	RRT*	153.5 $\pm$ 57.7	223 $\pm$ 13	444 $\pm$ 27	47.49 $\pm$ 1.56	14.00 $\pm$ 0.63	2977 $\pm$ 219
	RRG	289.8 $\pm$ 8.4	1500	26 826 $\pm$ 217	45.90 $\pm$ 0.67	15.80 $\pm$ 0.40	3549 $\pm$ 92
	Visibility	186.9	34	266	47.93	16.00	3595
	Cell decomp. (exact)	484.2	85	86	50.28	15.00	3490
	Voronoi / GVD	2211.6	1920	2843	49.82	16.00	3595
	Cell decomp. (approx.)	5054.7	21 279	41 910	45.09	15.00	3365

Table 3: Victims count experiment: 15 victims / 130s (exact Held-Karp) and 20 victims / 200s (greedy + 2-opt)

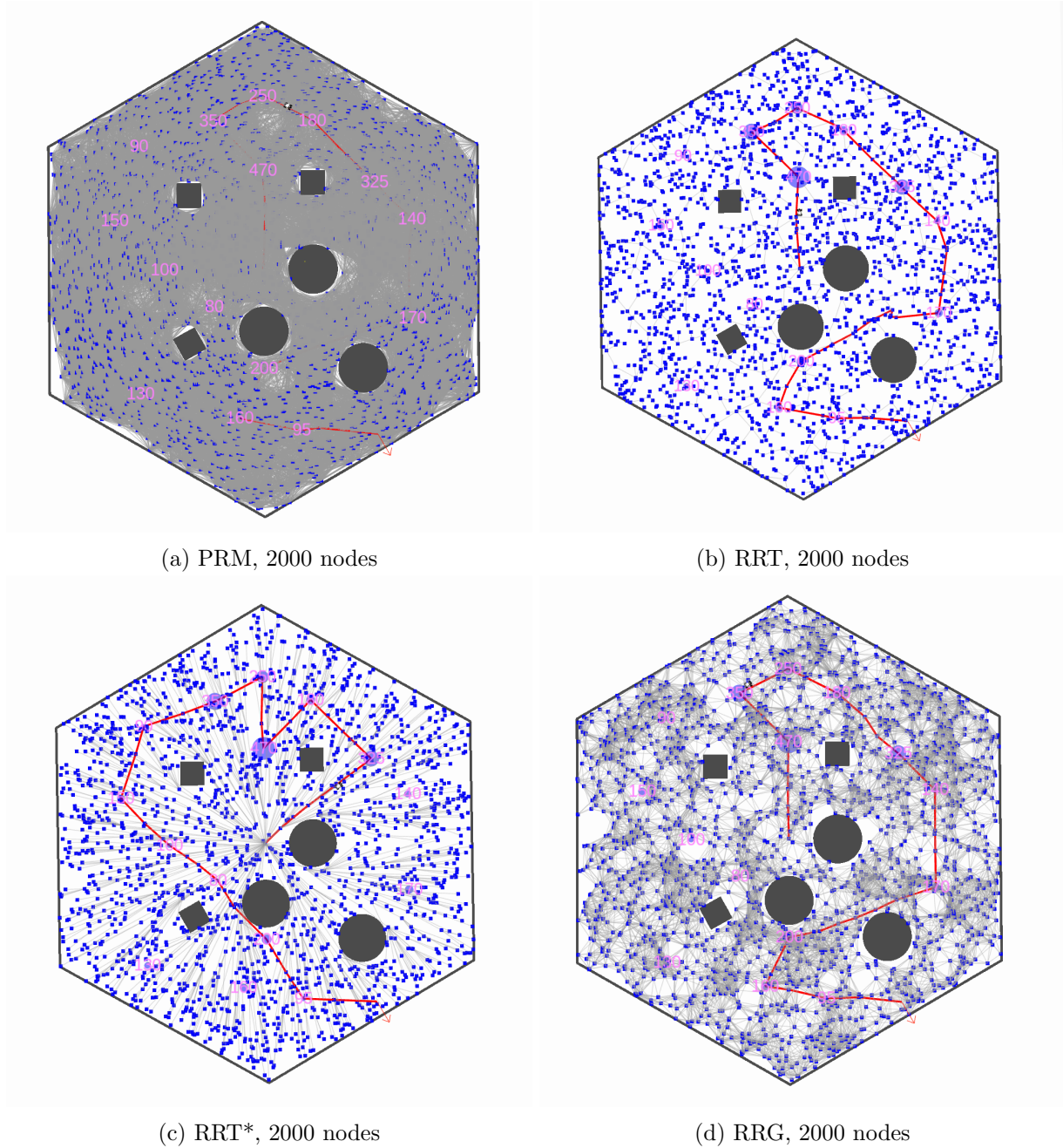


Figure 2: Comparison of roadmaps depending on the number of nodes – 2000 nodes

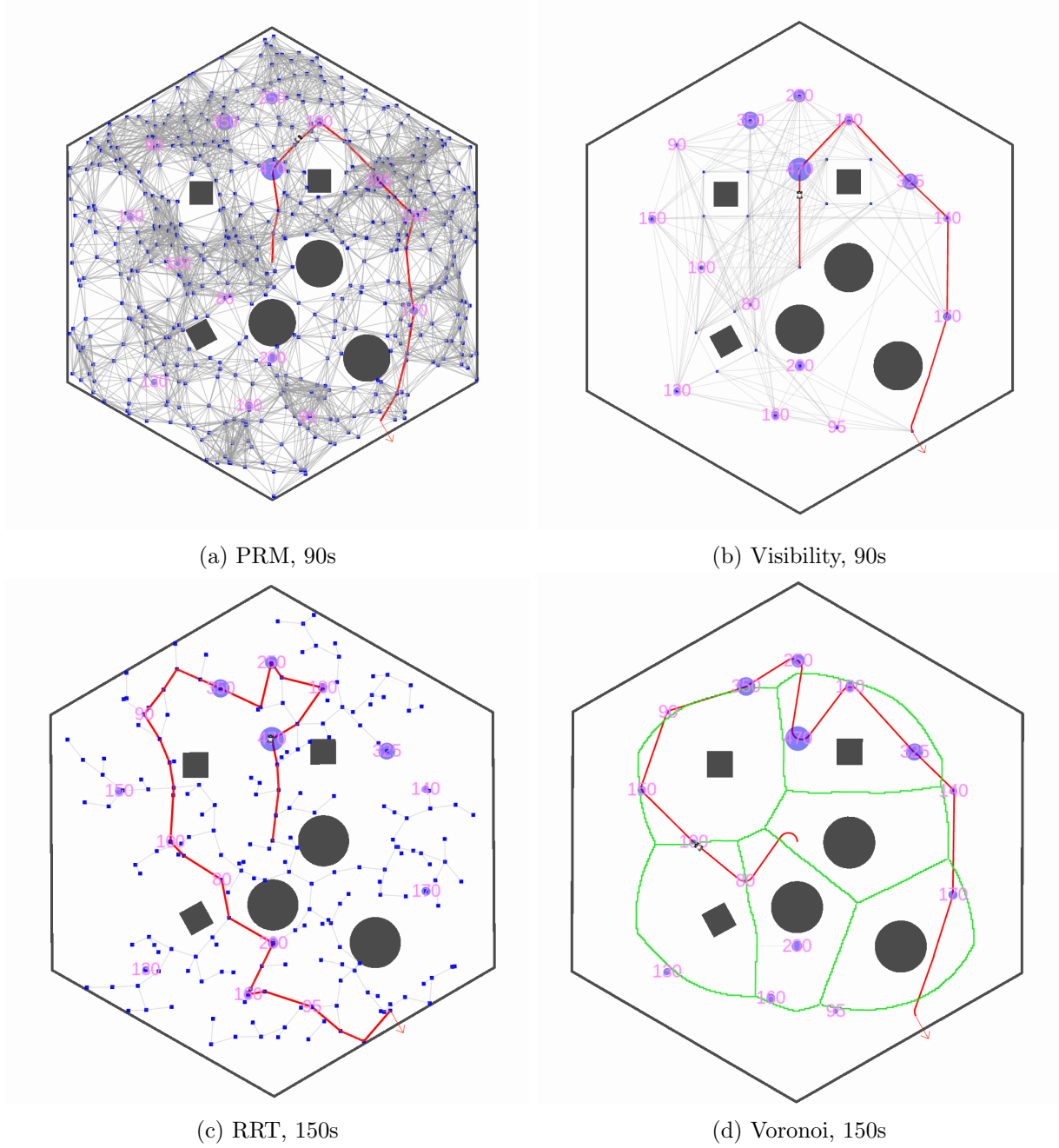


Figure 3: Time budget experiments (randomically selected images) – 1/2

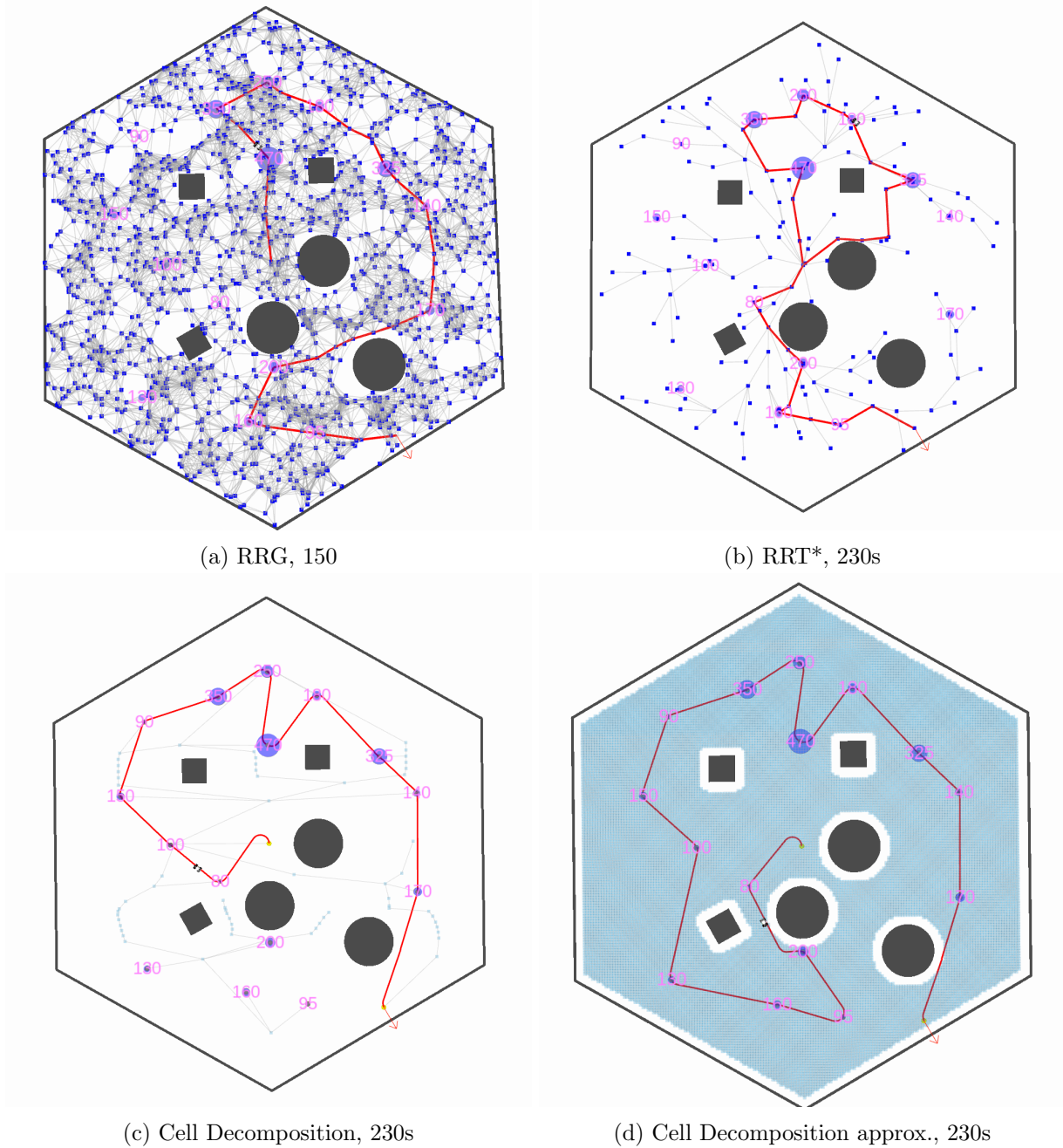


Figure 4: Time budget experiments (randomically selected images) – 2/2

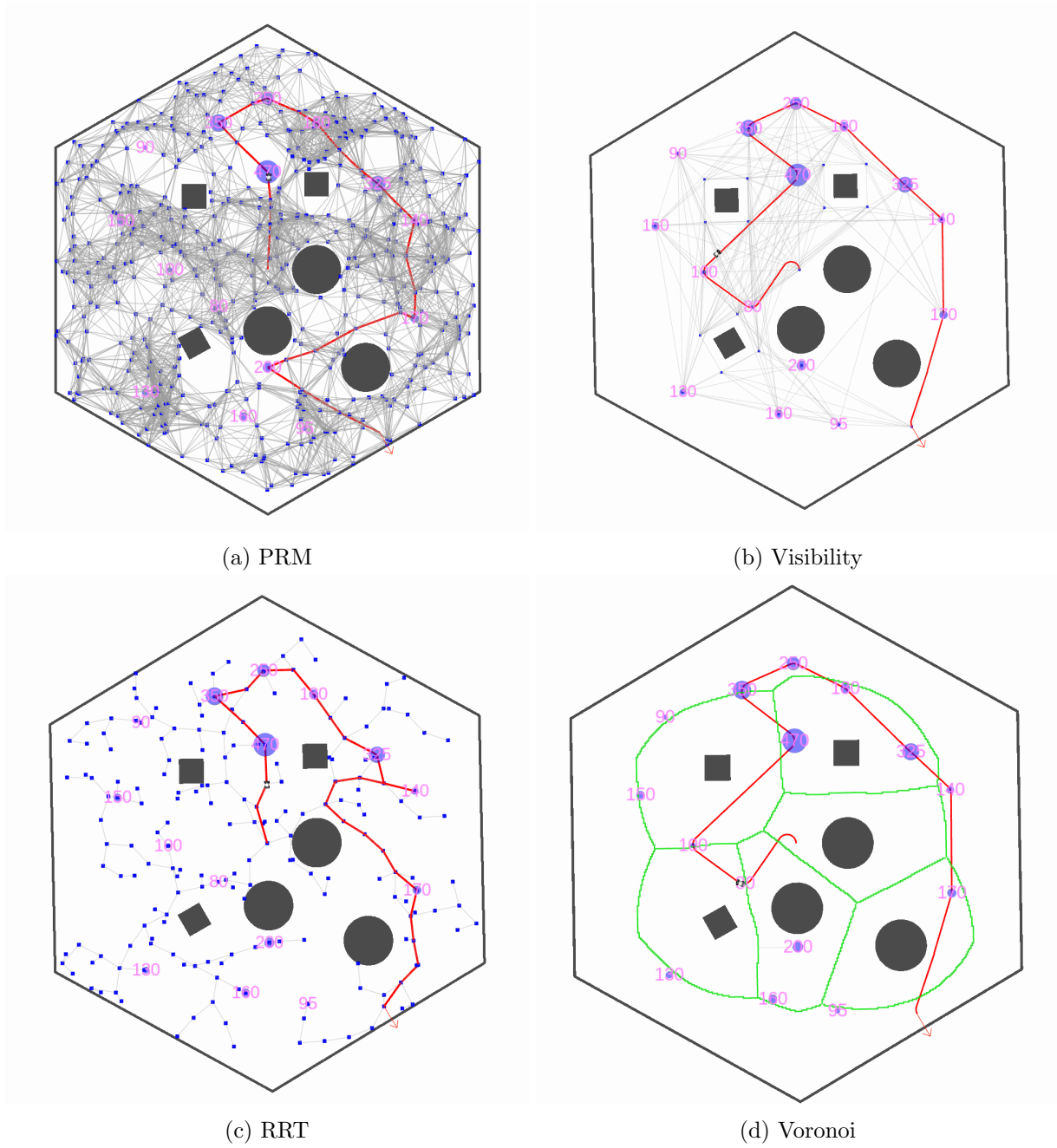


Figure 5: 15 victims experiment – 1/2

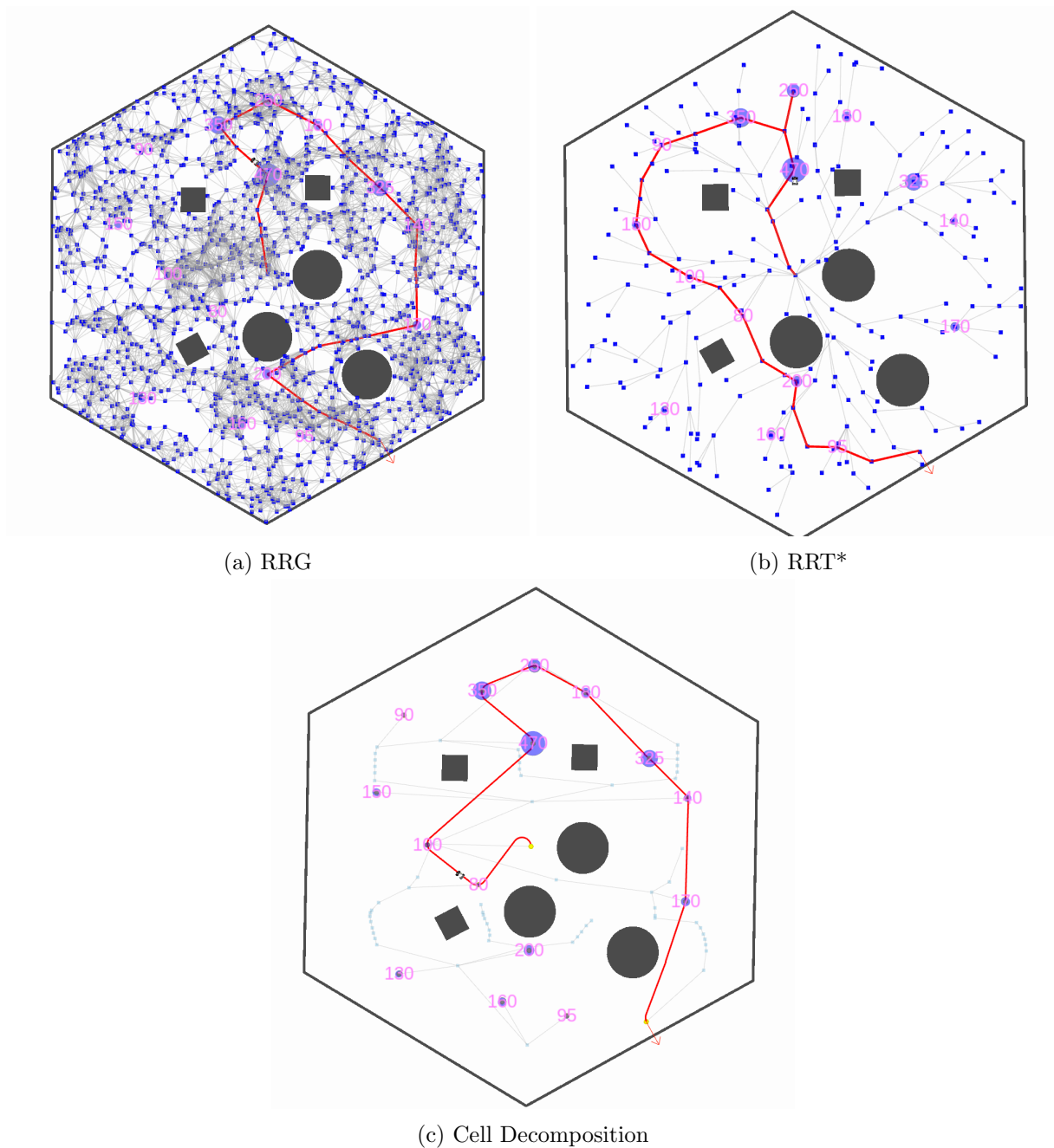


Figure 6: 15 victims experiment – 2/2

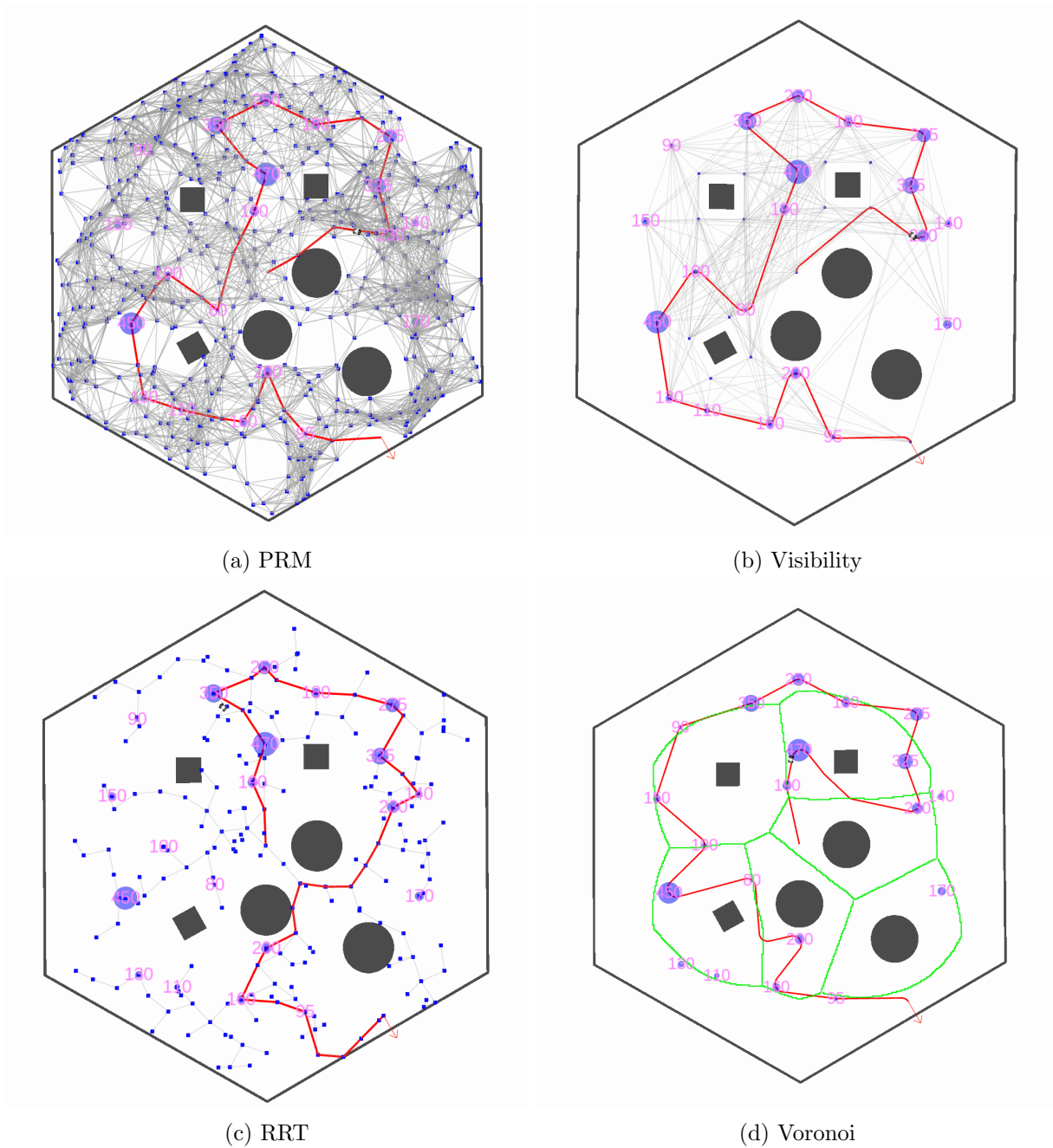


Figure 7: 20 victims experiment – 1/2

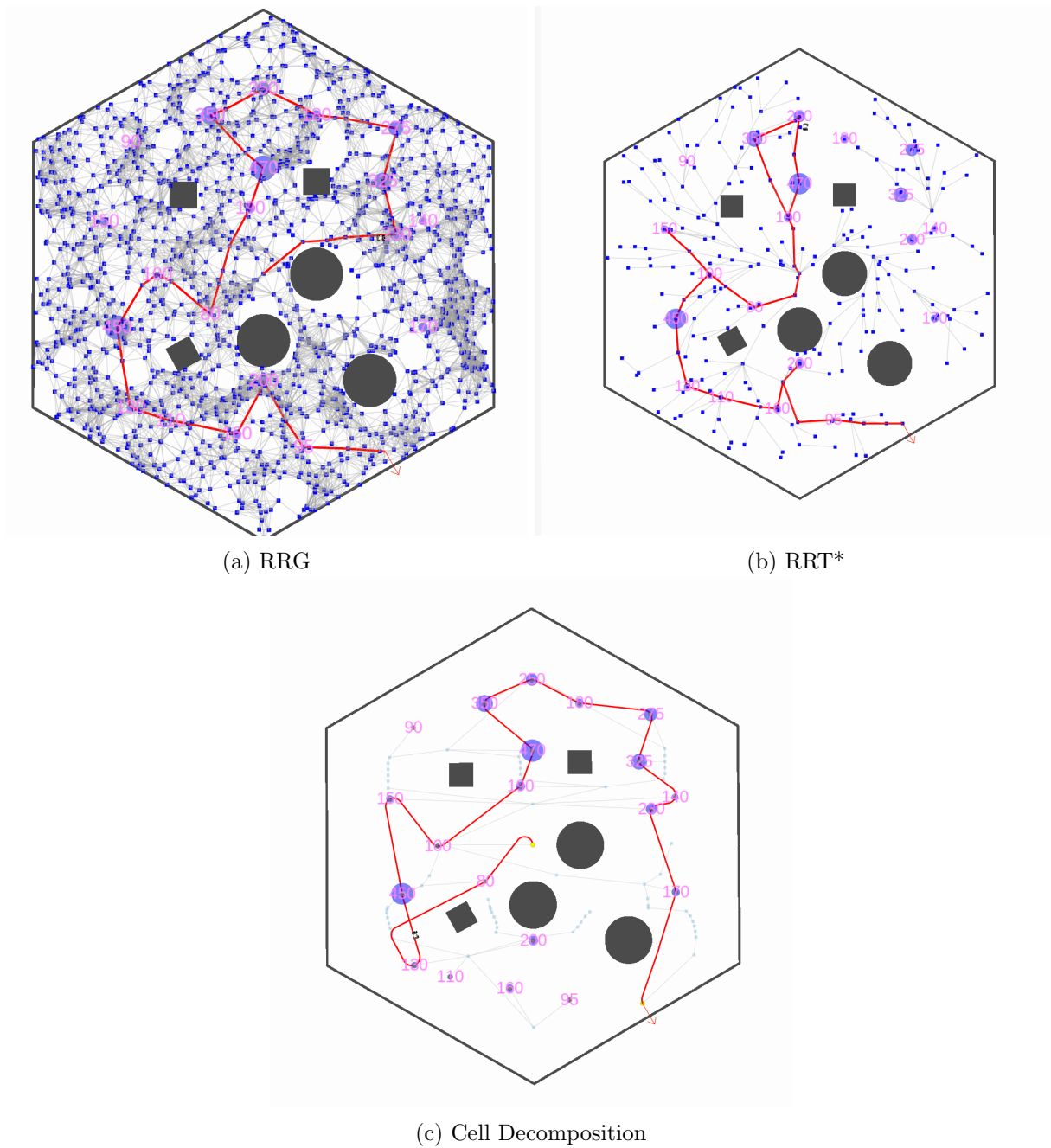


Figure 8: 20 victims experiment – 2/2

End of Updates

2 Introduction

2.1 Project Overview

This report describes the design and implementation of a motion-planning stack for a simulated rescue mission (*Target Rescue*), in which a non-holonomic ground robot must visit as many victims as possible, each carrying a scalar score reward, within a polygonal arena with obstacles of different shapes, respecting a global time budget, and finally reach the exit gate. The package implements and compares two families of planners sharing a common ROS interface: three *combinatorial* (roadmap-based) planners (an exact visibility graph, an exact/approximate cell decomposition, and a maximum-clearance Generalised Voronoi Diagram (GVD) roadmap) and three *sampling-based* planners (PRM, RRT and RRT*). All combinatorial strategies share a common downstream pipeline: the roadmap is collapsed into an all-pairs shortest-distance matrix between the points of interest via Dijkstra, victim selection and ordering is solved as an *Orienteering Problem* (exact Held-Karp dynamic programming, or a greedy + 2-opt heuristic for larger instances), and the resulting waypoint sequence is converted into a flyable, bounded-curvature trajectory by jointly optimising the intermediate headings and stitching Dubins manoeuvres, with an iterative clearance-repair loop. We describe the theory behind each roadmap strategy, the software architecture that lets every planner reuse the same mission and trajectory layers, and the benchmarking methodology used to compare them on path length, planning time, safety and victim value collected.

2.2 Scenario

This project implements the *Target Rescue* scenario provided for the Robot Planning course using the `loco_nav` simulation submodule. The environment is generated automatically at the beginning of each mission and contains:

- a polygonal **map border** and a set of polygonal or circular **obstacles**;
- a set of **victims**, each associated with a reward value;
- one or more exit **gates** with a required final orientation;
- a mission **time limit**, published through the `/victims.timeout` topic.

The robot starts from its initial pose and must rescue as many victims as possible before reaching the exit gate. The trajectory must remain collision-free, satisfy the robot's kinematic constraints and respect the available mission time.

Since each victim contributes a different reward, rescuing every victim is not always the best strategy. The planner must therefore decide not only how to move through the environment, but also which victims are worth visiting in order to **maximize the total collected value**.

2.3 Planning approaches

The project compares the two main families of robot motion planners presented during the course.

The first family consists of **combinatorial planners**, which build a deterministic roadmap directly from the geometry of the environment. Three different strategies are implemented:

- Visibility Graph;
- Cell Decomposition (exact and approximate variants);
- Generalized Voronoi Diagram (GVD).

The second family consists of **sampling-based planners**, which explore the free space by randomly generating valid robot configurations. Three planners are implemented:

- Probabilistic Roadmap (PRM);
- Rapidly-exploring Random Tree (RRT);
- Rapidly-exploring Random Tree Star (RRT*).

All six planners implement the same **Planner** interface and can therefore be executed by the same ROS node. Whenever possible, they also reuse the same software components for collision checking, trajectory generation and benchmarking, allowing the comparison to focus on the characteristics of the planning algorithms rather than on implementation differences.

3 Problem Formulation

3.1 Workspace and robot model

The robot operates in a two-dimensional workspace $\mathcal{W}$, which represents the map. The map is enclosed by a polygonal border and contains a set of obstacles $\mathcal{O} = \{O_1, \dots, O_m\}$, where each one can be either a polygon or a circular object.

The planning is simplified by treating the robot as a point. To do this, every obstacle is virtually enlarged by a safety clearance (see Section 5.1).

The robot follows the Dubins vehicle model: it moves forward at a constant speed and cannot turn more sharply than its minimum turning radius ρ (equivalently, maximum curvature $k_{\max} = 1/\rho$).

3.2 Victim selection as an Orienteering Problem

The map contains three types of points of interest:

- the start pose s of the robot,
- the exit gate g ,
- a set of victims $V = \{v_1, \dots, v_n\}$ with an associated reward score w_i .

The objective is to rescue as many valuable victims as possible while respecting the available mission time and still reaching the exit gate.

To make this decision, the planner first computes the shortest collision free distance between every pair of points of interest. We denote this distance by $D(a, b)$, where $a, b \in \{s\} \cup V \cup \{g\}$.

Once these distances are known, the mission becomes an **Orienteering Problem (OP)**: select which victims to visit and in which order in order to maximise the total collected reward without exceeding the available travel distance.

This is formulated as

$$\max_{k, \pi} \sum_{j=1}^k w_{\pi(j)} \quad \text{subject to} \quad D(s, v_{\pi(1)}) + \sum_{j=1}^{k-1} D(v_{\pi(j)}, v_{\pi(j+1)}) + D(v_{\pi(k)}, g) \leq D_{\max}, \quad (1)$$

where $D_{\max}$ is the maximum distance the robot can travel during the mission.

The distance budget is computed from the available mission time T , the robot speed $v_{\max}$ and a safety factor η_{safety} :

$$D_{\max} = v_{\max} T \eta_{\text{safety}}. \quad (2)$$

The safety factor leaves some margin because the final Dubins trajectory is usually longer than the shortest path computed on the roadmap.

3.3 Planning pipeline

The complete planning process is divided into four independent stages:

- **Geometry**: The map is built from the ROS topics containing the borders, obstacles, victims and gate. Collision checking is performed using the chosen safety clearance.
- **Roadmap generation**: A planner (Visibility Graph, Cell Decomposition, GVD, PRM, RRT or RRT*) computes a graph or search structure representing the collision-free space.
- **Mission planning**: The roadmap is used to compute the shortest distances between all points of interest using Dijkstra's algorithm. These distances form a matrix that is given to the Orienteering solver, which selects the victims to rescue and their visiting order.

- **Trajectory generation:** The selected sequence of waypoints is converted into a feasible trajectory for a Dubins vehicle by optimizing the headings at intermediate points and connecting them with Dubins curves. The resulting trajectory is checked again for collisions before execution.

Dividing the problem into these stages makes the software modular. The different roadmap planners implemented can be compared while reusing the same mission planning and trajectory generation components.

4 Software Architecture

4.1 The Planner interface

All planning algorithms implement the same `Planner` interface. This allows the rest of the software to use any planner without changing the surrounding code.

```

1 class Planner {
2 public:
3     virtual void initialize(const WorldModel& world) = 0;
4     virtual void step() = 0;
5     virtual void visualize() = 0;
6     virtual bool isPlanningDone() const = 0;
7     void setMetrics(RunMetrics* m);
8 protected:
9     RunMetrics* metrics_ = nullptr;
10 };

```

Each function has a specific role:

- `initialize()` receives the shared `WorldModel`, which contains the map and all the information needed for planning.
- `step()` is called periodically by the ROS node. Once the world has been completely received, it performs the planning algorithm.
- `visualize()` publishes RViz markers such as the roadmap, search tree and final trajectory.
- `isPlanningDone()` tells the benchmark program when the planner has finished, so that the execution statistics can be recorded.

Because every planner follows the same interface, switching between different algorithms simply requires creating a different planner object.

4.2 The WorldModel

The `WorldModel` is a shared data structure containing all the information received from the simulator through ROS topics.

It stores:

- the map borders,
- the obstacles,
- the victim positions and their associated rewards,
- the gate position,
- the robot's initial pose,
- the available mission time.

It also contains a set of “ready” flags that indicate when all the required information has been received. This prevents a planner from starting before the environment has been completely initialized.

4.3 Shared planning pipeline

The three combinatorial planners (Visibility Graph, Cell Decomposition and Voronoi) all follow the same planning pipeline shown in Figure 9.

The only stage that changes between planners is the roadmap construction. Once the roadmap has been generated, the remaining steps are identical and are implemented only once inside `task/tour_builder.cpp`. This modular design allows different roadmap planners to reuse exactly the same mission planning and trajectory generation code, avoiding duplicated code and making the different planners easier to compare.

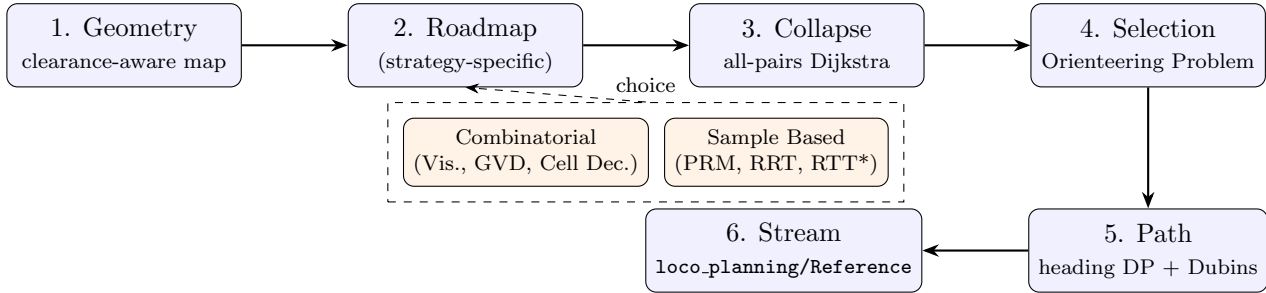


Figure 9: Shared planning pipeline used by the combinatorial planners. Only the roadmap generation stage depends on the chosen algorithm, while all the following stages are shared.

The pipeline works as follows:

1. A clearance aware representation of the environment is built.
2. A roadmap is generated using the selected planning strategy.
3. Dijkstra’s algorithm computes the shortest distances between all points of interest.
4. The Orienteering Problem solver selects which victims to rescue and in which order.
5. The resulting sequence is converted into a feasible Dubins trajectory.
6. The trajectory is published as reference poses for the robot controller.

4.4 ROS nodes

The project contains two main ROS executables.

- `map_receiver` is used during normal execution. It receives the environment information, runs the selected planner and publishes the reference trajectory for the robot.
- `benchmark` performs the same planning process but automatically collects performance statistics, allowing different planners to be compared under identical conditions.

Both nodes subscribe to the topics containing the map, obstacles, victims, gate, robot odometry and mission time. The planner to execute is selected through the ROS parameter `planner_type`, making it possible to switch between Visibility Graph, Cell Decomposition, Voronoi, PRM, RRT and RRT* without modifying the code.

Once planning is complete, the generated trajectory is published on the `/<robot>/ref` topic as a sequence of `loco_planning/Reference` messages, which are followed by the controller provided by `loco_nav`. The benchmark node also collects in a `.csv` summary statistics such as planning time, collected reward, etc.

5 Planning Strategies

The project implements two families of motion planners. The first family consists of **combinatorial planners**, which construct a deterministic roadmap directly from the geometry of the environment. The second family consists of **sampling-based planners**. Instead of explicitly representing the free space, these algorithms randomly sample valid robot configurations and progressively build either a roadmap or a search tree. Their probabilistic nature makes them suitable for complex environments, although different executions may produce different solutions.

All planners use the same collision-checking module, ensuring that the comparison focuses on the planning strategy itself rather than on differences in the geometric representation. Once the roadmap has been built, all planners reuse the same mission-planning pipeline described in Section 4: the shortest paths between the points of interest are computed, the victims to rescue are selected by solving the Orienteering Problem, and the final Dubins trajectory is generated.

5.1 Shared collision checking

All planners use the same clearance aware collision checker, this guarantees that all planners operate under the same geometric assumptions, making their performance directly comparable.

The robot is treated as a point by enlarging every obstacle with a safety clearance

$$c = r_{\text{robot}} + \delta_{\text{safety}},$$

where r_{robot} is the robot radius and δ_{safety} is an additional safety margin.

Instead of explicitly computing enlarged obstacle polygons using Minkowski sums, the implementation simply checks whether a point lies inside an obstacle or closer than the clearance distance from its boundary. The same approach is used for the map borders.

To determine whether a path segment is collision free, the segment is sampled at regular intervals. If every sampled point is valid, the segment is considered traversable.

5.2 Combinatorial planners

Combinatorial planners build an explicit roadmap from the obstacle geometry before any path search is performed. The roadmap is deterministic and completely determined by the map, meaning that repeated executions always produce the same graph.

Although the roadmap construction differs between planners, they all produce the same output: a graph containing the start position, the victims, the gate and any additional roadmap nodes. This graph is then processed by the shared mission-planning pipeline described in the previous section.

5.2.1 Visibility graph

The Visibility Graph is the classical exact roadmap for polygonal environments. Its nodes consist of the robot start, the gate, all victim locations and the clearance-adjusted obstacle vertices.

Two nodes are connected whenever the straight segment between them is completely collision-free. Since shortest paths around polygonal obstacles always pass through obstacle vertices, the resulting graph contains the true shortest path between any pair of points of interest.

The roadmap is built by testing the visibility between every pair of candidate nodes. Although this requires a quadratic number of visibility tests, the number of vertices in the considered maps is relatively small, making the construction fast enough for the project.

The Visibility Graph is deterministic, complete and produces graph-optimal shortest paths. However, the generated paths tend to pass close to obstacles, maintaining only the required safety clearance.

5.2.2 Cell decomposition

Cell decomposition partitions the free space into connected regions and represents each region with a roadmap node. Two nodes are connected whenever their corresponding cells share a boundary, allowing the robot to move from one cell to the other.

Two variants are implemented:

Exact vertical decomposition The workspace is divided into vertical slabs generated from the obstacle geometry. Inside each slab, the free space is identified and represented by one or more cells. Adjacent cells belonging to neighbouring slabs are connected whenever their free intervals overlap.

Because every obstacle feature is considered during the decomposition, this method is complete: whenever a collision-free path exists, a corresponding path also exists in the roadmap.

Approximate grid decomposition The second implementation discretizes the environment into a regular square grid. A grid cell is considered free only if its centre and corners are collision-free. Adjacent free cells are then connected using 4-connectivity.

Unlike the exact decomposition, this approximation is not complete. Narrow passages may disappear if the grid resolution is too coarse, preventing the planner from finding an existing path. The advantage is a much simpler implementation whose computational cost depends mainly on the chosen grid resolution.

5.2.3 Generalized Voronoi Diagram (GVD)

The Generalized Voronoi Diagram (GVD) follows a different objective from the previous planners. Instead of minimizing the travelled distance, it attempts to maximize the distance from nearby obstacles, producing safer trajectories.

The roadmap is computed on a regular occupancy grid. First, an Euclidean distance transform assigns every free cell its distance from the nearest obstacle. Cells that are approximately equidistant from two or more obstacles form the Voronoi skeleton. This skeleton is then thinned and converted into a graph, while the start position, victims and gate are connected to the closest reachable skeleton nodes.

Because the implementation is based on a discrete grid, the resulting roadmap is only an approximation of the continuous Voronoi diagram. The generated paths usually have a larger clearance than those of the Visibility Graph, but they are also longer and may fail to preserve connectivity if the grid resolution is too coarse.

5.3 Sampling-based planners

Sampling-based planners avoid constructing an exact geometric roadmap. Instead, they explore the free space by generating random collision-free configurations.

Unlike the combinatorial planners, these algorithms are probabilistic: two executions on the same map may generate different roadmaps or trees and therefore different solutions. Increasing the number of samples generally improves the quality of the solution at the cost of additional computation.

5.3.1 Probabilistic Roadmap (PRM)

PRM is the sampling-based counterpart of the combinatorial roadmap planners.

A fixed number of collision-free configurations are sampled uniformly inside the free space. Each sample becomes a roadmap node, and nearby nodes are connected whenever the straight segment joining them is collision-free. The resulting graph can then be searched using Dijkstra's algorithm to compute shortest paths.

Since the roadmap is generated randomly, different executions may produce slightly different graphs. Increasing the number of samples improves connectivity and path quality, but also increases the roadmap construction time.

5.3.2 Rapidly-exploring Random Tree (RRT)

Unlike PRM, RRT does not build a complete roadmap before searching for a solution. Instead, it incrementally grows a tree starting from the robot's initial position.

At each iteration, a random configuration is sampled inside the workspace. The nearest node already present in the tree is found, and a new node is generated by moving a fixed step toward the sampled point. The new edge is added only if it is collision-free.

The tree naturally expands toward unexplored regions of the environment, allowing the planner to quickly discover feasible paths even in large search spaces. As additional nodes are generated, the planner can progressively reach new victims and eventually connect the tree to the exit gate.

5.3.3 Rapidly-exploring Random Tree Star (RRT*)

RRT* extends RRT by improving the quality of the generated tree.

Whenever a new node is added, the algorithm searches the neighbouring nodes and selects the parent that minimizes the travel cost from the start. It then attempts to reconnect nearby nodes

through the new node whenever this reduces their cost. This rewiring process gradually improves the overall tree.

Unlike standard RRT, whose first solution may remain far from optimal, RRT* is asymptotically optimal: as the number of samples increases, the generated solution converges toward the optimal one. This improved path quality comes at the cost of a higher computational effort for each iteration.

5.4 Planner comparison

Tables 4 and 5 summarize the main theoretical properties of the six planning strategies considered.

Table 4: Comparison of the combinatorial planners.

Property	Visibility	Cell Decomp.	Voronoi / GVD
Compl.	Yes (exact)	Yes (exact) / No (approx.)	No (grid-discrete)
Opt.	Shortest on graph	Sub-optimal	Sub-optimal
Safety	Clearance only	Clearance only	Maximum clearance
Det.	Yes	Yes	Yes
Complexity	$O(V^2S)$	$O(MH/\text{res} + C^2)$	$O(N \log N)$
Parameter	<code>node_buffer</code>	<code>sample_res</code> , <code>cell_size</code>	<code>voronoi_res</code> , <code>thin_iters</code>

Table 5: Comparison of the sampling-based planners.

Property	PRM	RRT	RRT*
Compl.	Prob. complete	Prob. complete	Prob. complete
Opt.	Sub-optimal	Not optimal	Asymptotically optimal
Safety	Clearance only	Clearance only	Clearance only
Det.	No	No	No
Complexity	$O(nk)$	$O(n \log n)^*$	$O(n \log n)^*$
Parameter	Samples, radius	Step size	Step size, rewiring radius

*Typical complexity when using efficient nearest-neighbour data structures.

6 Mission Planning: the Orienteering Problem

Once a roadmap has been generated, the planning problem is no longer purely geometric. At this stage, the planner already knows how to move between different locations in the environment. The remaining task is to decide which victims should be rescued and in which order while respecting the available mission time.

This problem is modelled as an Orienteering Problem and is solved independently of the roadmap generation strategy.

6.1 Distance matrix computation

The first step is to compute the shortest path between every pair of points of interest, namely the robot start position, all victims and the exit gate.

For the combinatorial planners, Dijkstra’s algorithm is executed once from each point of interest on the generated roadmap. Besides the shortest distances, the predecessor information is also stored so that the corresponding roadmap path can later be reconstructed.

The resulting distance matrix provides the input for the mission-planning stage.

6.2 Exact solver

For maps containing a moderate number of victims (typically up to 16), the Orienteering Problem is solved exactly using a Held–Karp dynamic programming algorithm.

Each dynamic programming state stores the minimum travel distance required to visit a particular subset of victims and finish at a given victim. The algorithm progressively expands these subsets until every feasible solution has been considered.

Among all routes that satisfy the available distance budget, the one collecting the largest total reward is selected. If two solutions collect the same reward, the shorter one is preferred.

Although the algorithm has an exponential complexity of

$$O(2^n n^2),$$

it remains practical for the problem sizes considered in this project and guarantees the optimal victim selection.

6.3 Greedy heuristic

For larger instances, solving the Orienteering Problem exactly becomes computationally expensive. A faster heuristic is therefore provided.

Starting from an empty route, victims are inserted one at a time in the position that provides the highest reward compared to the additional travel distance introduced by the insertion. This process continues until no further victim can be added without exceeding the available budget.

The resulting route is then improved using the well-known 2-opt local search algorithm, which repeatedly exchanges route segments whenever this shortens the total path length.

Although this heuristic does not guarantee the optimal solution, it has polynomial complexity and produces good solutions in a much shorter time.

The planner automatically selects between the exact and heuristic solver depending on the number of victims, although either method can also be chosen explicitly through the `op_method` parameter.

6.4 Waypoint reconstruction

Once the visiting order has been determined, the solution is converted back into a geometric path.

For every consecutive pair of points of interest, the corresponding shortest roadmap path is reconstructed using the predecessor information previously computed by Dijkstra's algorithm. These path segments are then concatenated into a single sequence of waypoints, removing duplicated nodes where consecutive segments meet.

The resulting waypoint sequence is passed to the trajectory generation stage, where it is converted into a collision-free Dubins trajectory that can be executed by the robot.

7 Trajectory Generation: Heading Optimization and Dubins Paths

The output of the mission-planning stage is an ordered sequence of waypoints connecting the robot start, the selected victims and the exit gate. Since the robot cannot instantaneously change its heading, this waypoint sequence must be converted into a feasible trajectory that satisfies the vehicle's kinematic constraints.

7.1 Dubins paths

The robot is modelled as a Dubins vehicle, which moves at constant forward speed and has a minimum turning radius ρ . As a result, the robot cannot follow arbitrary piecewise-linear paths.

For every pair of consecutive waypoints, the planner computes the shortest Dubins path connecting the corresponding poses. The six classical Dubins path types (LSL, RSR, LSR, RSL, RLR and LRL) are evaluated, and the shortest feasible one is selected.

7.2 Heading optimization

The waypoint positions are fixed by the mission planner, but only the robot heading at the start and at the exit gate is predefined. The headings at the intermediate waypoints can instead be freely chosen. The goal is to select these headings so that the total length of the stitched Dubins trajectory is minimized.

To solve this problem, each intermediate heading is discretized into a finite number of candidate orientations (72 by default). Dynamic programming is then used to compute the combination of headings producing the minimum total Dubins length.

The dynamic programming algorithm treats each waypoint and candidate heading as a state. The transition cost between two consecutive states is simply the length of the corresponding Dubins path. By processing the waypoints sequentially, the algorithm finds the globally shortest sequence of headings.

To improve the solution quality, each waypoint is also initialized with headings aligned with the direction of the neighbouring waypoints. These additional candidates often produce better trajectories even when using a relatively small heading discretization.

Algorithm 1 Heading optimization using dynamic programming

Require: waypoints $p_0, \dots, p_{P-1}$; fixed θ_0, θ_{P-1} ; heading discretization D

- 1: Generate candidate headings for every waypoint
 - 2: Initialize the first state with the start heading
 - 3: **for** every waypoint **do**
 - 4: **for** every candidate heading **do**
 - 5: Compute the Dubins cost from every candidate of the previous waypoint
 - 6: Keep the transition with minimum accumulated cost
 - 7: **end for**
 - 8: **end for**
 - 9: Backtrack the optimal heading sequence
-

7.3 Collision verification

Although the roadmap guarantees that the straight segments connecting consecutive waypoints are collision-free, the corresponding Dubins curves may deviate from these segments and intersect nearby obstacles.

For this reason, every generated Dubins path is sampled and checked again using the same collision-checking module employed during roadmap construction.

If a collision is detected, an additional waypoint is inserted halfway along the corresponding roadmap segment. The heading optimization is then repeated using the refined waypoint sequence. This process is iterated until a collision-free trajectory is obtained or a maximum number of refinement steps is reached.

7.4 Trajectory publication

Once the final trajectory has been validated, it is discretized using a constant time step while assuming constant robot speed, as required by the project specification.

Each trajectory sample contains the robot position, heading, linear velocity, angular velocity and timestamp. These samples are published as `loco_planning/Reference` messages on the `/<robot>/ref` topic, allowing the controller provided by `loco_nav` to execute the planned trajectory.

8 Implementation Details

8.1 Package organization

The planner is implemented as the ROS package `rescue_planner`. Its structure follows the modular architecture presented in Section 4, separating the different stages of the planning pipeline into dedicated modules.

The main directories are:

- `include/planning/` and `src/planning/`: implementations of the six planning algorithms (Visibility Graph, Cell Decomposition, Voronoi, PRM, RRT and RRT*);
- `include/task/` and `src/task/`: shared mission-planning components, including roadmap utilities, the Orienteering solver and the common algorithms reused by multiple planners;

- `include/trajectory/` and `src/trajectory/`: Dubins path generation and heading optimization;
- `include/utils/` and `src/utils/`: common utility functions, including collision checking and benchmarking support;
- `src/map_receiver.cpp` and `src/benchmark.cpp`: the two executable ROS nodes used for interactive execution and automated benchmarking;
- `launch/`: launch files used to start the simulation and select the desired planning algorithm.

8.2 Configuration parameters

Most planner parameters can be configured through ROS without recompiling the project. Table 6 summarizes the main parameters used by the planners (defaults as set in the constructors).

Table 6: ROS parameters of the planners

Parameter	Default	Meaning
<code>robot_radius</code>	0.20 m	physical robot radius
<code>safety_margin</code>	0.10 m	additional clearance margin
<code>v_max</code>	0.30 m/s	constant cruise speed
<code>turning_radius</code>	0.35 m	ρ ; $k_{\max} = 1/\rho$
<code>victim_radius</code>	0.50 m	geometric victim disk radius
<code>dt</code>	0.01 s	reference discretisation step
<code>dubins_safety</code>	0.85	budget derating factor η_{safety}
<code>sample_res</code>	0.05 m	collision sampling resolution
<code>node_buffer</code>	clearance+0.10 m	(visibility only) corner offset
<code>cell_size</code>	0.10 m	(approx. cell decomp. only) grid pitch
<code>voronoi_res</code>	0.05 m	(Voronoi only) distance-transform grid pitch
<code>thin_iters</code>	20	(Voronoi only) max. Zhang–Suen iterations
<code>dubins_discretizations</code>	72	heading candidates per free waypoint
<code>op_method</code>	auto	auto/exact/greedy OP solver selector

In addition to the planner parameters, the simulation environment can be configured through the `map_config.yaml` file provided by the `loco_nav` framework. This file controls the characteristics of the generated scenario, including:

Table 7: Main simulation parameters defined in `loco_nav`’s `map_config.yaml`.

Parameter	Example	Meaning
<code>victims_timeout</code>	40 -140 s	Mission time budget
<code>n_victims</code>	4	Number of randomly generated victims
<code>n_obstacles</code>	3	Number of randomly generated obstacles
<code>no_cylinders</code>	true / false	Enable or disable cylindrical obstacles
<code>no_boxes</code>	true / false	Enable or disable box-shaped obstacles
<code>vect_x, vect_y</code>	lists	Fixed obstacle or victim positions (if provided)

In particular, the `victims_timeout` parameter determines the available mission time and was used to evaluate the planners under different time budgets. The same configuration file also allows fixed obstacle and victim positions to be specified instead of randomly generating the environment. This proved useful during development and debugging, since the same scenario could be reproduced across multiple executions and planning algorithms.

8.3 Benchmarking

The **benchmark** node automatically records the planner initialization time, planning time, mission parameters, roadmap size, trajectory length, collected reward, number of rescued victims and the final mission outcome. The collected data are stored in a CSV file, making it possible to compare the different planners over multiple randomly generated maps.

8.4 Testing

The project also includes a set of unit tests covering the roadmap-independent components of the software, including the Orienteering solver, the Dubins path generator and the heading optimization algorithm. Testing these modules independently from ROS and Gazebo made it possible to validate the mission-planning and trajectory-generation stages before integrating them with the different planning algorithms.

9 Experimental Setup

9.1 Simulation environment

All experiments are performed in the `loco_nav` Gazebo simulator using the same `limo` differential-drive robot and the map generator provided with the course.

To ensure a fair comparison, all planners are evaluated on the same map configuration for each experiment. The map contains four obstacles (a mixture of boxes and cylinders), one exit gate and the same set of 3 victims. Only the parameters under evaluation are modified between experiments.

9.2 Evaluation metrics

For each execution we record the following quantities:

- planning time;
- time budget;
- roadmap size (number of nodes and edges, when applicable);
- total trajectory length;
- number of rescued victims;
- total reward collected;
- mission success (whether the robot reaches the exit gate within the available time).

These values are automatically collected by the benchmarking node and stored in a CSV file for later analysis.

9.3 Experimental protocol

The evaluation consists of three groups of experiments.

1. **Mission duration.** Each planner is executed on the same map using three different mission time limits: (40 s, 70 s, 140 s). This allows us to evaluate how the available time influences victim selection and trajectory generation.
2. **Sampling-based planners.** PRM is additionally tested with 200 and 1000 sampled nodes, and RRT/RRT* with 300 and 2500 sampled nodes, in order to study the effect of the roadmap/tree density on the solution quality and planning time.
3. **Combinatorial planners.** For the combinatorial approaches, the main planner-specific parameters are varied (such as the visibility graph corner offset or the grid resolution for the decomposition-based planners) to evaluate their influence on the generated roadmap and the resulting trajectory.

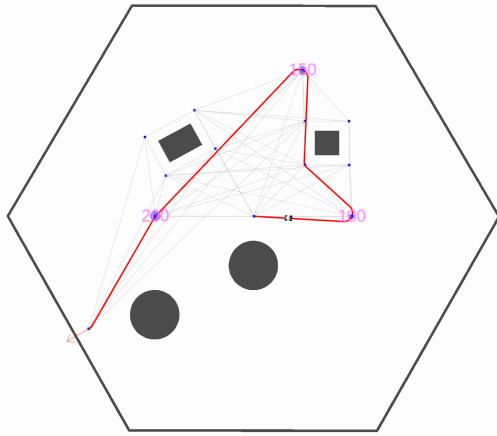
Whenever a planner contains random sampling (PRM, RRT and RRT*), multiple executions are performed for each configuration and the average performance is reported. The deterministic combinatorial planners always produce the same solution on a fixed map and therefore require only one execution per configuration.

10 Results

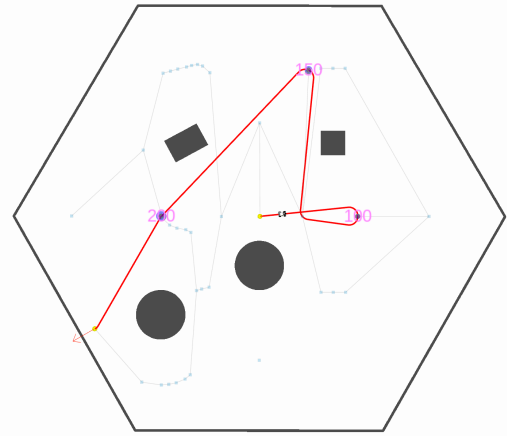
This section reports the empirical comparison across the six planners on the same map described in Section 9 (four obstacles, one gate, three victims), collected through the `benchmark.csv` logs produced by the benchmarking node. Table 8 reports, for each of the three mission time budgets, the mean planning time, roadmap/tree size, flyable path length and mission outcome for every planner; for PRM, RRT and RRT*, being stochastic planners, three independent runs were executed per configuration and the path length is reported as mean $\pm$ standard deviation, while the three combinatorial planners, being fully deterministic, required a single run each.

Table 8: Benchmark results

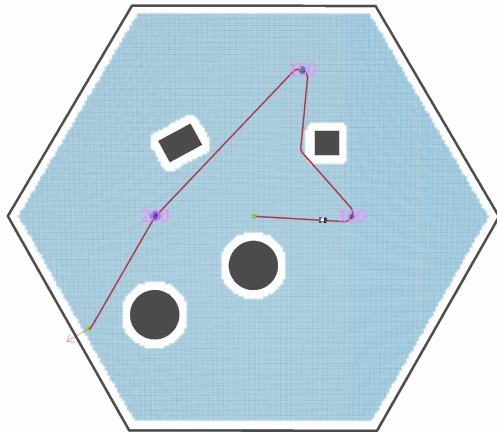
Budget	Planner	Plan (ms)	Nodes	Edges	Path (m)	Victims	Score
40 s	Visibility	13.7	13	50	10.49	1.00	200.0
	Cell decomp. (exact)	26.5	40	40	10.48	1.00	200.0
	Cell decomp. (approx)	684.7	22034	43518	10.48	1.00	200.0
	Voronoi / GVD	406.8	1602	2447	10.48	1.00	200.0
	PRM	90.2	500	10648	9.38 $\pm$ 0.06	1.00	200.0
	RRT	82.6	1500	2998	9.49 $\pm$ 0.17	0.33	66.7
	RRT*	120.9	1500	2998	9.54 $\pm$ 0.13	1.00	200.0
70 s	Visibility	21.7	13	50	17.72	2.00	300.0
	Cell decomp. (exact)	43.2	40	40	17.72	2.00	300.0
	Cell decomp. (approx)	674.7	22034	43518	17.72	2.00	300.0
	Voronoi / GVD	395.3	1602	2447	17.72	2.00	300.0
	PRM	138.2	500	10777	17.47 $\pm$ 0.12	2.00	300.0
	RRT	100.8	1500	2998	10.36 $\pm$ 0.29	1.00	200.0
	RRT*	144.4	1500	2998	12.57 $\pm$ 4.26	1.33	233.3
140 s	Visibility	38.3	13	50	24.88	3.00	450.0
	Cell decomp. (exact)	48.2	40	40	26.73	3.00	450.0
	Cell decomp. (approx)	735.1	22034	43518	24.83	3.00	450.0
	Voronoi / GVD	523.6	1602	2447	26.97	3.00	450.0
	PRM	176.6	500	10825	24.74 $\pm$ 0.04	3.00	450.0
	RRT	242.6	1500	2998	30.96 $\pm$ 2.20	3.00	450.0
	RRT*	1201.9	1500	2998	29.82 $\pm$ 0.71	3.00	450.0



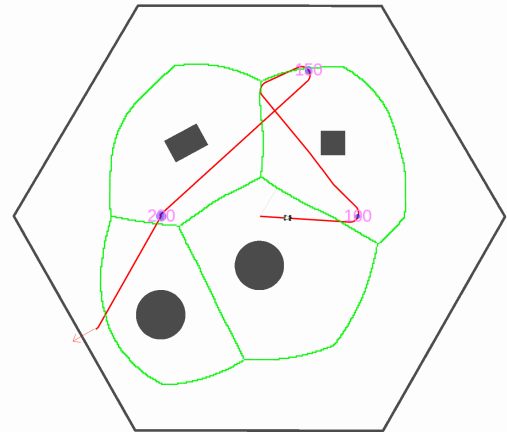
(a) Visibility graph



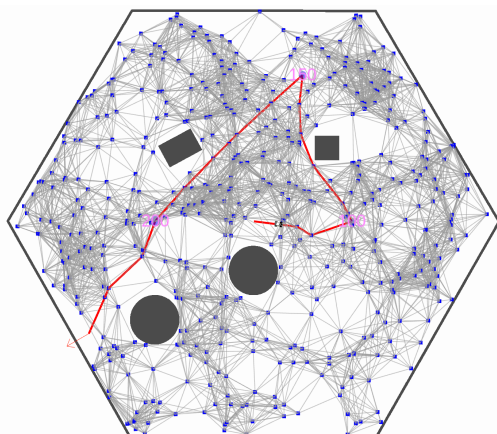
(b) Cell decomp. (exact)



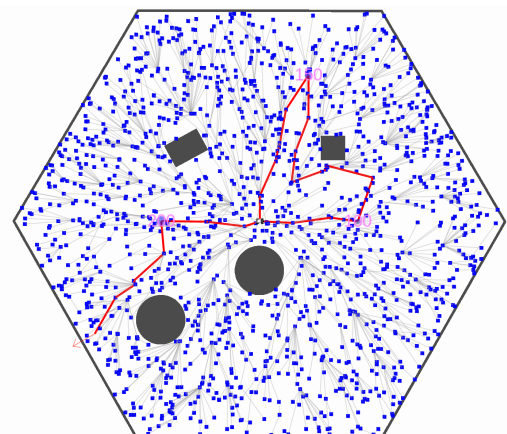
(c) Cell decomp. (approx.)



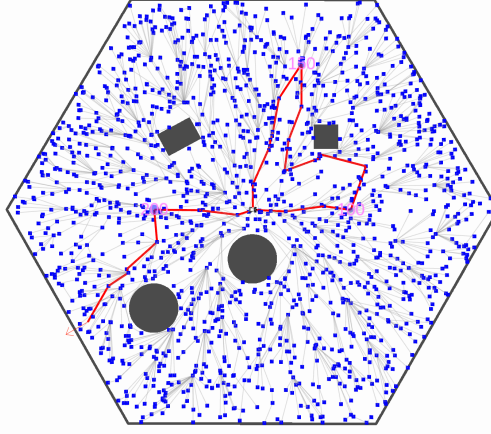
(d) Voronoi / GVD



(e) PRM, 500 nodes



(f) RRT, 1500 nodes



(a) RRT*, 1500 nodes

Figure 11: RViz screenshots at the 140 s budget. The four combinatorial roadmaps make the theoretical contrast of Section 5 directly visible: the visibility graph and the exact cell decomposition are sparse, the Voronoi skeleton traces a dense mid-clearance ridge through the whole free space, and the approximate grid decomposition covers the entire map at fine resolution. The sampling-based runs, by contrast, show the irregular, run-dependent coverage typical of randomised exploration.

Observations

The Table 8 results with the theoretical predictions of seen in the course yield the following observations.

Victim selection is robust to the roadmap strategy. At every budget, all three combinatorial planners and, whenever it succeeds, every sampling-based planner too, converge on the *same* number of victims and the same score. The choice of *which* victims to rescue, driven entirely by the shared Orienteering-Problem solver (Section 6), is essentially decoupled from the underlying roadmap topology on this map: the differences between planners show up almost exclusively in *how* the chosen tour is traversed (path length, planning time), not in *what* is chosen.

The visibility graph is confirmed shortest, but the margin is small. At 140 s, so with three victims, when the tour most likely to interact with the obstacles, the visibility graph gives the shortest flyable path (24.88 m), closely followed by the approximate cell decomposition (24.83 m, marginally shorter), with the exact cell decomposition and the Voronoi roadmap both longer (26.73 m and 26.97 m). At 40 s and 70 s the four combinatorial paths are essentially indistinguishable (within 0.02 m), since a short, mostly obstacle-free tour barely exercises the differences between the four roadmap topologies.

The approximate cell decomposition is expensive to build. The fine grid resolution used in the simulations produces the largest graph of the four combinatorial (22 034 nodes, 43 518 edges) and consistently the *slowest* planning time of all seven planners (675–735 ms). The resolution/build-cost trade-off is in this specific case unbalanced.

Sampling-based planners show the variance predicted in Section 5. RRT’s path length standard deviation reaches ± 2.20 m at 140 s and RRT*’s planning time spikes to 3.08 s in one of its three 140 s runs (against 245–283 ms in the other two) – both symptomatic of the stochastic, run-dependent behaviour that the deterministic combinatorial planners never exhibit. Under the tightest budget (40 s), plain RRT even fails to rescue *any* victim in one of its three runs (score 0), while PRM and RRT* remained reliably successful.

RRT* improves over RRT, but the rewiring cost is steep. At 140 s, RRT*’s mean path length (29.82 m) is shorter than RRT’s (30.96 m), consistent with its asymptotic-optimality guarantee, but at roughly $5\times$ the mean planning time (1202 ms vs. 243 ms). This is a clear illustration of the optimality/computation trade-off between the two tree-based planners.

Denser sampling helps, and can be necessary for completeness. In the dedicated node-

count experiments, increasing PRM’s sample count from 200 to 1000 shortens its path from 26.90 m to 24.40 m, and increasing RRT’s from 300 to 2500 shortens it from 34.55 m to 31.33 m. RRT* is the starkest case: with only 300 samples it fails to connect one of the three victims within budget (2 victims, score 350), while with 2500 samples it recovers all three (score 450).

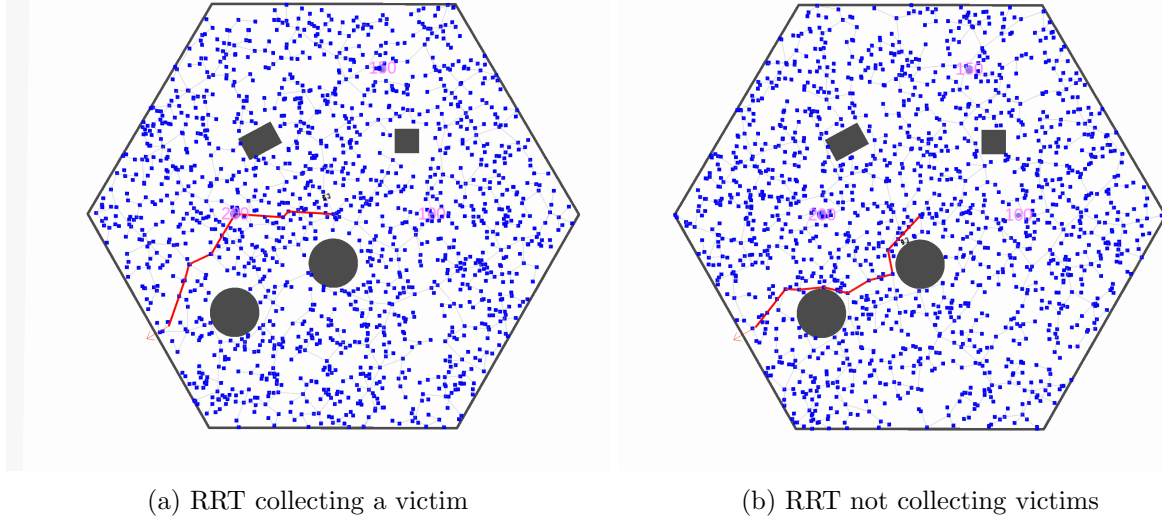


Figure 12: RRT with 40s time budget showing the random sampling might rescue or not a victim in different runs.

11 Discussion

11.1 Path Quality, Completeness and Safety

The experimental results largely confirm the theoretical trade-offs discussed in Section 5. As expected, the visibility graph consistently produces the shortest trajectories, while the Voronoi roadmap generates paths that visibly maintain a larger clearance from obstacles, albeit at the expense of longer travel distances. The exact cell decomposition represents an intermediate solution, producing trajectories that are slightly longer than those of the visibility graph while preserving the completeness guarantees of an exact decomposition.

An interesting observation concerns the approximate cell decomposition. Despite its lack of theoretical completeness, the fine grid resolution adopted in the experiments allows it to produce trajectories whose length is nearly identical to that of the visibility graph (24.83m and 24.88m for the 140s mission). This result does not contradict the optimality guarantees of the visibility graph. Rather, it shows that a sufficiently dense discretisation can closely approximate the same shortest path solution, although at the cost of a substantially larger roadmap and significantly higher computational effort.

11.2 Deterministic and Sampling-Based Planning

The experimental results clearly highlight one of the fundamental differences between the two planning paradigms. As expected, the combinatorial planners are fully deterministic: given the same environment, they always construct the same roadmap and therefore produce identical rescue trajectories, resulting in zero run-to-run variation. In contrast, the sampling-based planners exhibit the inherent variability associated with random sampling. Both RRT and RRT* show noticeable variations in path length and planning time across repeated executions.

From a practical perspective, this distinction is particularly relevant in rescue scenarios subject to strict mission constraints. For deterministic planners, the behaviour observed in a single execution is representative of all future executions on the same map. Conversely, the average performance reported for sampling-based planners may conceal occasional unsuccessful runs, such as the RRT execution that failed to collect any victim within the available budget. Consequently, evaluating

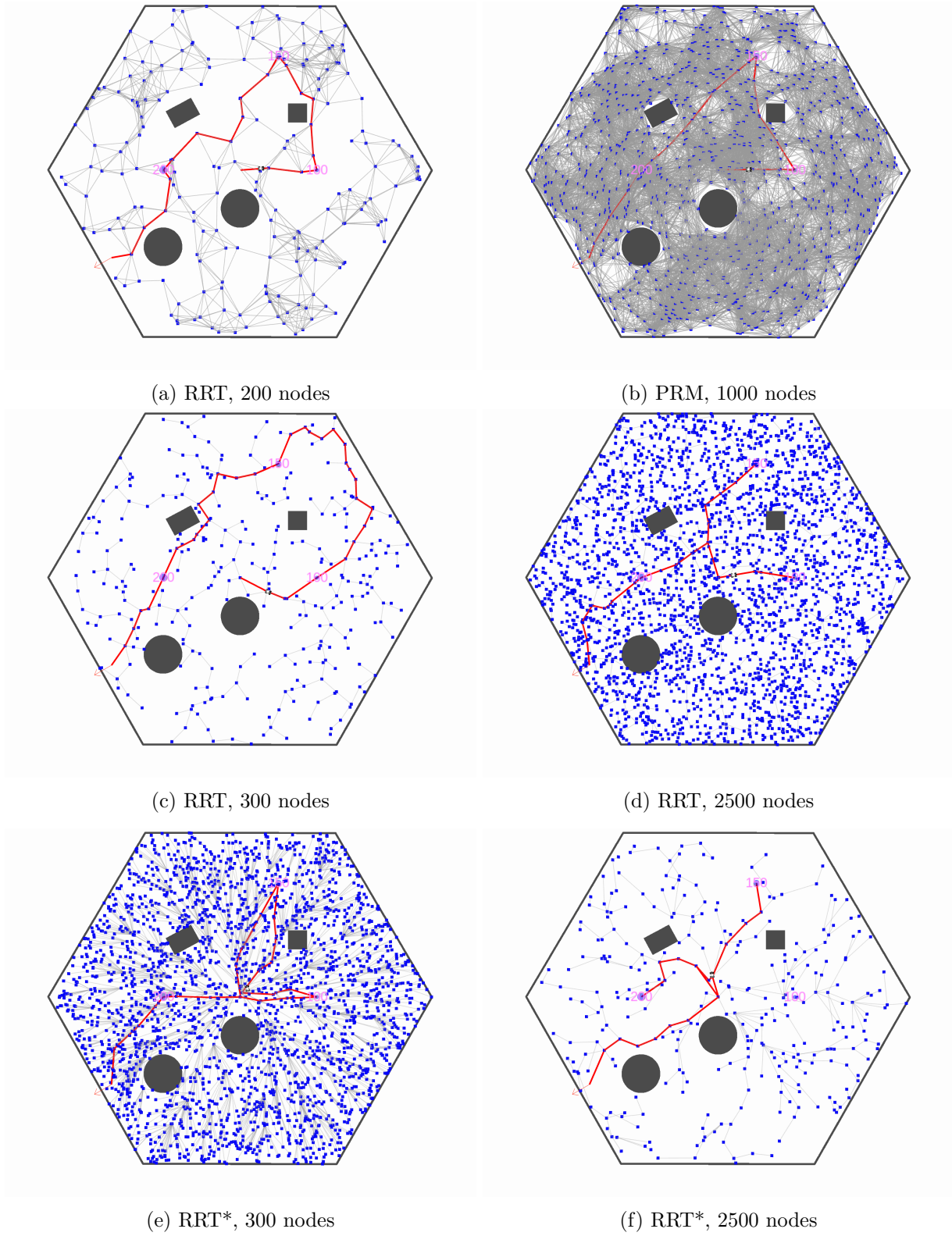


Figure 13: Comparison of roadmaps depending on the number of nodes

an insufficient number of performances may underestimate or overestimate the impact of stochastic failures in applications.

11.3 Influence of Resolution and Sample Density

The experiments also illustrate the influence of the main tuning parameter for each planning family. For the combinatorial planners, the grid resolution adopted by the approximate cell decomposition directly determines the fidelity of the free-space representation. A sufficiently fine discretisation produces path lengths comparable to those of the exact planners, but substantially increases roadmap construction time.

A similar behaviour is observed for the sampling-based planners, where the number of sampled configurations governs both roadmap quality and computational cost. The dedicated parameter sweep shows that insufficient sampling density may prevent the planner from discovering feasible connections, whereas increasing the number of samples improves both solution quality and mission success. In particular, increasing the sample count from 300 to 2500 allows RRT* to consistently recover feasible solutions while reducing the resulting trajectory length.

Overall, these results indicate that neither grid resolution nor sample density should be regarded as simple performance parameters. Instead, they directly affect the planner's ability to represent the connectivity of the free space, and therefore determine the trade-off between computational cost and solution quality.

11.4 Optimality and Computational Cost in tree-based planners

The comparison between RRT and RRT* confirms the expected effect of the rewiring strategy introduced by RRT*. Across the evaluated scenarios, RRT* consistently generates shorter trajectories than standard RRT, while also exhibiting lower variability in path quality. These improvements, however, come at the expense of additional computation due to the neighbourhood search and rewiring operations performed after each node insertion.

In the tested rescue scenarios, this increased computational cost remains acceptable, as all sampling-based planners complete the planning stage well before the available mission time expires. Nevertheless, the occasional multi-second planning spike observed for RRT* suggests that its worst-case behaviour may become significant in applications requiring strict real-time replanning. The choice between RRT and RRT* therefore depends on the relative importance assigned to path quality and planning latency within the target application.

11.5 Limitations and Future Works

The benchmarking framework currently records only planning-related metrics and mission statistics. While qualitative inspection of the generated trajectories confirms that the Voronoi roadmap naturally maintains larger obstacle clearance than the other planners, this observation was not supported by a quantitative clearance metric. Extending the benchmarking framework to record the minimum obstacle clearance along each executed trajectory would provide a more complete assessment of the trade-off between path optimality and safety, and represents a natural direction for future works.

References

- [1] Project repository: https://github.com/Allelallecialle/Robot_Planning_FinalProject.git
- [2] Course submodule: https://github.com/idra-lab/loco_nav.git
- [3] RRG: "Incremental Sampling-based Algorithms for Optimal Motion Planning", S. Karaman and E. Frazzoli, 2010, <https://arxiv.org/abs/1005.0416>
- [4] "Following the university guidelines, LLMs were used as an additional and assistive tool for improving the report's language quality and the code implementation."